

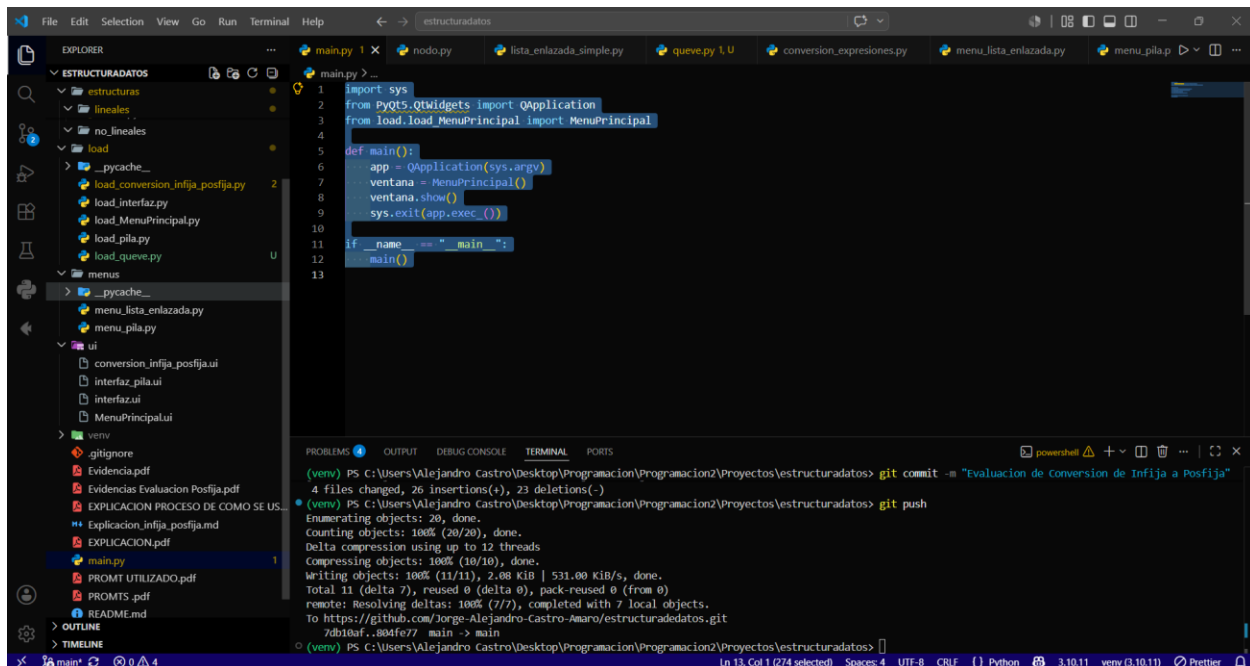
Actua como asistente de programacion para una practica de Estructura de Datos en Python. Contexto: Ya tengo un proyecto desarrollado en clase con Python y programacion orientada a objetos. En el proyecto ya existe una clase Stack o Pila y dos aplicaciones que utilizan esta estructura. Ahora necesito agregar una nueva estructura de datos: Cola o Queue. Objetivo: Necesito crear una clase Queue con las operaciones basicas enqueue, dequeue, printQueue, firstQueue y lastQueue. La clase debe integrarse al mismo proyecto existente sin modificar lo que ya funciona. Instrucciones importantes: 1. No generes un proyecto nuevo desde cero. 2. No reemplaces ni modifiques la clase Stack existente. 3. No elimines las aplicaciones ya desarrolladas con pila. 4. Respeta la organizacion actual de archivos, clases y nombres del proyecto. 5. La solucion debe estar hecha con programacion orientada a objetos. 6. Propón cambios minimos e indica exactamente en que archivo y en que parte deben colocarse. 7. La clase Queue debe tener al menos los metodos enqueue, dequeue, printQueue, firstQueue y lastQueue. 8. Si se requiere un metodo auxiliar para saber si la cola esta vacia, proponlo sin afectar la lista de operaciones solicitadas. 9. Explica brevemente como funciona FIFO y como se diferencia de una pila. 10. Proporciona pruebas para comprobar que la clase funciona correctamente. Codigo existente:

```
from estructuras.lineales.lista_enlazada_simple import LinkedList
class OperacionesPila:
    def __init__(self, lista: LinkedList):
        self.lista = lista
        self.resultado = ""
    def pila_agregar(self, data):
        self.lista.insert_at_beginning(data)
        self.resultado = f"Elemento {data} agregado a la pila."
    return self.resultado
    def pila_eliminar(self):
        eliminado = self.lista.delete_first()
        if eliminado is None:
            self.resultado = "La pila está vacía, no se puede eliminar."
        else:
            self.resultado = f"Elemento {eliminado} eliminado de la pila."
        return eliminado
    def pila_tope(self):
        if self.lista.head is None:
            self.resultado = "La pila está vacía."
        return None
        self.resultado = f"El tope de la pila es: {self.lista.head.data}"
        return self.lista.head.data
    def pila_imprimir(self):
        if self.lista.head is None:
            self.resultado = "La pila está vacía."
        return self.resultado
    elementos = []
    temp = self.lista.head
    while temp is not None:
        elementos.append(str(temp.data))
        temp = temp.next
    self.resultado = " -> ".join(elementos)
    return self.resultado
    def is_empty(self):
        return self.lista.head is None
class Queue(object):
    def __init__(self):
        self.first = None
        self.last = None
    def enqueue(self, valor):
        # Agrega el valor al final de la cola
        pass
    def dequeue(self):
        # Elimina el primer elemento que entro
        if self.isEmpty():
            return None
        pass
    def firstQueue(self):
        # Consulta el primer elemento sin eliminarlo
        if self.isEmpty():
            return None
        pass
    def lastQueue(self):
        # Consulta el ultimo elemento sin eliminarlo
        if self.isEmpty():
            return None
        pass
    def printQueue(self):
        # Devuelve el contenido completo de la cola
        pass
    def isEmpty(self):
        return self.fist == None'
[Pega aqui la estructura de archivos del proyecto](imagen adjuntada)
import sys
from PyQt5.QtWidgets import QApplication
from load.load_MenuPrincipal import MenuPrincipal
def main():
    app = QApplication(sys.argv)
    ventana = MenuPrincipal()
    ventana.show()
    sys.exit(app.exec_())
if __name__ == "__main__":
    main()

```

Resultado esperado de tu respuesta: - Primero analiza la estructura del codigo existente. - Despues propone la clase Queue. - Luego indica como integrarla al proyecto sin afectar Stack ni las aplicaciones existentes. - Finalmente proporciona una prueba guiada de enqueue, dequeue, printQueue, firstQueue y lastQueue. - No omitas explicacion, porque debo entender el codigo para defenderlo en clase.

Imagen que se adjunto al código:



RESPUESTA GENERADA POR LA IA:

Muy buena observación, Alejandro 🐼. Te aclaro la diferencia porque aquí es donde suele haber confusión:

🔗 ¿Por qué usé `LinkedList` y no la pila (stack)?

- La **pila (Stack)** está diseñada para trabajar con el principio **LIFO** (*Last In, First Out*). → Solo permite insertar y eliminar por un extremo (el tope).
- La **cola (Queue)** en cambio sigue el principio **FIFO** (*First In, First Out*). → Inserta por un extremo (final) y elimina por el otro (inicio).

Si intentaras construir la cola sobre la pila, estarías forzando operaciones que no corresponden:

- Para hacer `enqueue` tendrías que apilar elementos.
- Para hacer `dequeue` tendrías que desapilar hasta llegar al primero que entró, lo cual rompe la eficiencia y la semántica de la cola.

👉 Por eso, lo correcto es que **tanto la pila como la cola se apoyen en la misma estructura base: la lista enlazada (LinkedList)**.

- La pila usa `insert_at_beginning` y `delete_first`.
- La cola usa `insert_at_end` y `delete_first`.

De esa forma, **no duplicas código** y mantienes coherencia en tu proyecto: una sola estructura (`LinkedList`) y encima de ella diferentes abstracciones (`OperacionesPila`, `Queue`).

✓ Resumen para defenderlo en clase

- **No se usa la pila para construir la cola**, porque son estructuras con reglas distintas.
- Ambas se apoyan en la **lista enlazada simple** como base.
- La pila controla el acceso por un extremo (tope).
- La cola controla el acceso por dos extremos (inicio y final).
- Esto respeta el principio de **modularidad**: una sola estructura base (`LinkedList`) y diferentes controladores que le dan semántica.

🔗 Ejemplo rápido para que lo veas claro

```
python
# Pila (Stack) → LIFO
pila = OperacionesPila(LinkedList())
pila.pila_agregar(1)
pila.pila_agregar(2)
print(pila.pila_imprimir()) # 2 -> 1
print(pila.pila_eliminar()) # 2 (último en entrar, primero en salir)

# Cola (Queue) → FIFO
cola = Queue()
cola.enqueue(1)
cola.enqueue(2)
print(cola.printQueue()) # 1 -> 2
print(cola.dequeue()) # 1 (primero en entrar, primero en salir)
```

☞ Entonces, la cola **no debe construirse sobre la pila**, sino directamente sobre la lista enlazada, igual que la pila. ¿Quieres que te arme un diagrama sencillo (tipo capas) para mostrar en clase cómo `Node` → `LinkedList` → `Stack/Queue` se relacionan?

Prueba utilizada en la terminal:

```
from estructuras.lineales.queue import Queue
```

```
def menu():
    print("\n=== MENÚ DE PRUEBA PARA COLA ===")
    print("1. Enqueue (agregar elemento)")
    print("2. Dequeue (eliminar elemento)")
    print("3. FirstQueue (consultar primero)")
```

```
print("4. LastQueue (consultar último)")
print("5. PrintQueue (mostrar cola)")
print("6. IsEmpty (verificar si está vacía)")
print("0. Salir")
```

```
def test_queue_interactivo():
```

```
    cola = Queue()
```

```
    opcion = -1
```

```
    while opcion != 0:
```

```
        menu()
```

```
        try:
```

```
            opcion = int(input("Selecciona una opción: "))
```

```
        except ValueError:
```

```
            print("Debes ingresar un número válido.")
```

```
            continue
```

```
    if opcion == 1:
```

```
        valor = input("Ingresa el valor a encolar: ")
```

```
        cola.enqueue(valor)
```

```
        print(cola.resultado)
```

```
    elif opcion == 2:
```

```
        eliminado = cola.dequeue()
```

```
        print(cola.resultado)
```

```
    elif opcion == 3:
```

```
        primero = cola.firstQueue()
```

```
        print(cola.resultado)
```

```
elif opcion == 4:
```

```
    ultimo = cola.lastQueue()
```

```
    print(cola.resultado)
```

```
elif opcion == 5:
```

```
    print("Contenido de la cola:", cola.printQueue())
```

```
elif opcion == 6:
```

```
    if cola.isEmpty():
```

```
        print("La cola está vacía.")
```

```
    else:
```

```
        print("La cola tiene elementos.")
```

```
elif opcion == 0:
```

```
    print("Saliendo del programa...")
```

```
else:
```

```
    print("Opción no válida, intenta de nuevo.")
```

```
if __name__ == "__main__":
```

```
    test_queue_interactivo()
```